	Лекции	**Практики**
Неделя 8 (19-22.10)	Темы 2-4	РГР
Неделя 9 (26-29.10)	Темы 5-7	РГР
Неделя 10 (2-5.11)	-	ЛР 8 (Темы 1-4)
Неделя 11 (9-12.11)	Темы 8-10	ЛР 9 (Темы 5-7)
Неделя 12 (16-19.11)	Темы 11-12	ЛР 10 (Темы 8-10)
Неделя 13 (23-26.11)	Темы 13-15	ЛР 11 (Темы 11-12)
Неделя 14 (30.11-3.12)	Темы 16-17	ЛР 12 (Темы 13-15)
Неделя 15 (7-10.12)	Темы 18-20	ЛР 13 (Темы 16-17)
Неделя 16 (14-17.12)	Тест по курсу DEV-1	ЛР 14 (Темы 18-20)

Организация данных
Логическая структура



Темы

Базы данных и шаблоны

Схемы и путь поиска

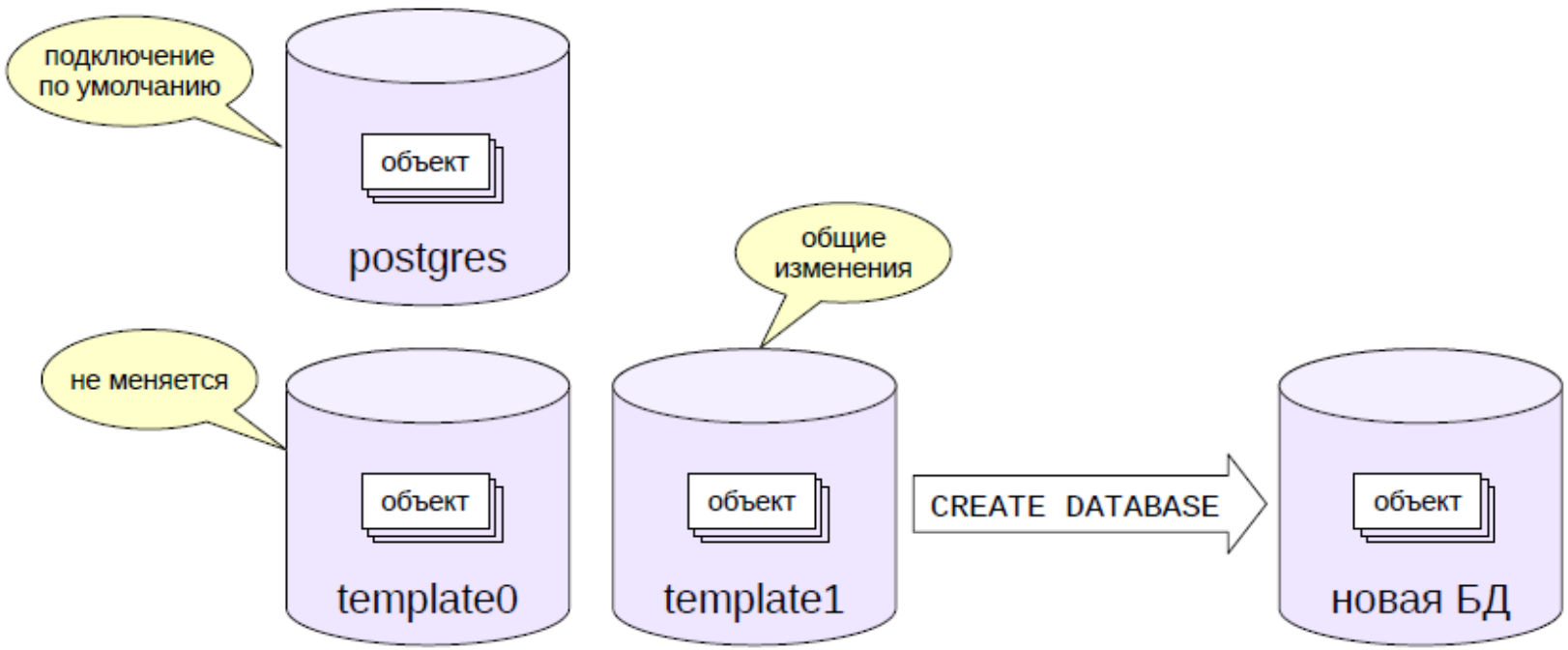
Специальные схемы

Системный каталог

Кластер баз данных

Инициализация кластера создает три базы данных

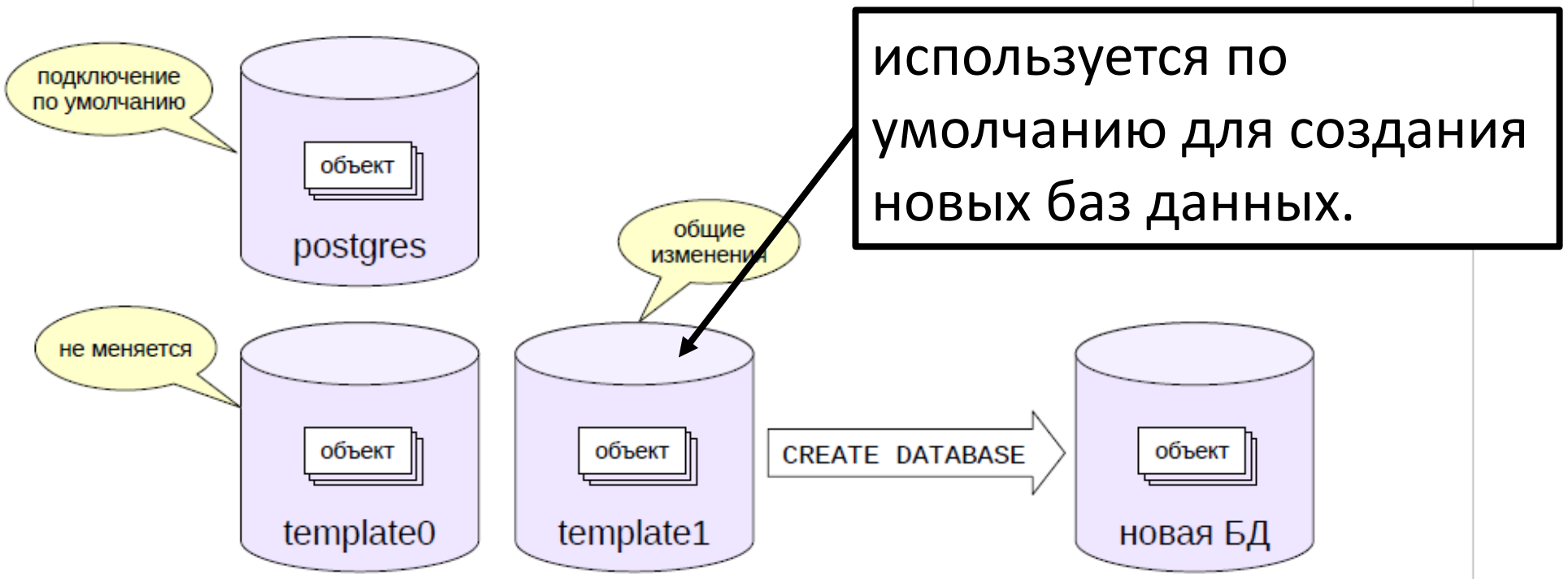
Новая база всегда клонируется из существующей



Кластер баз данных

Инициализация кластера создает три базы данных

Новая база всегда клонируется из существующей

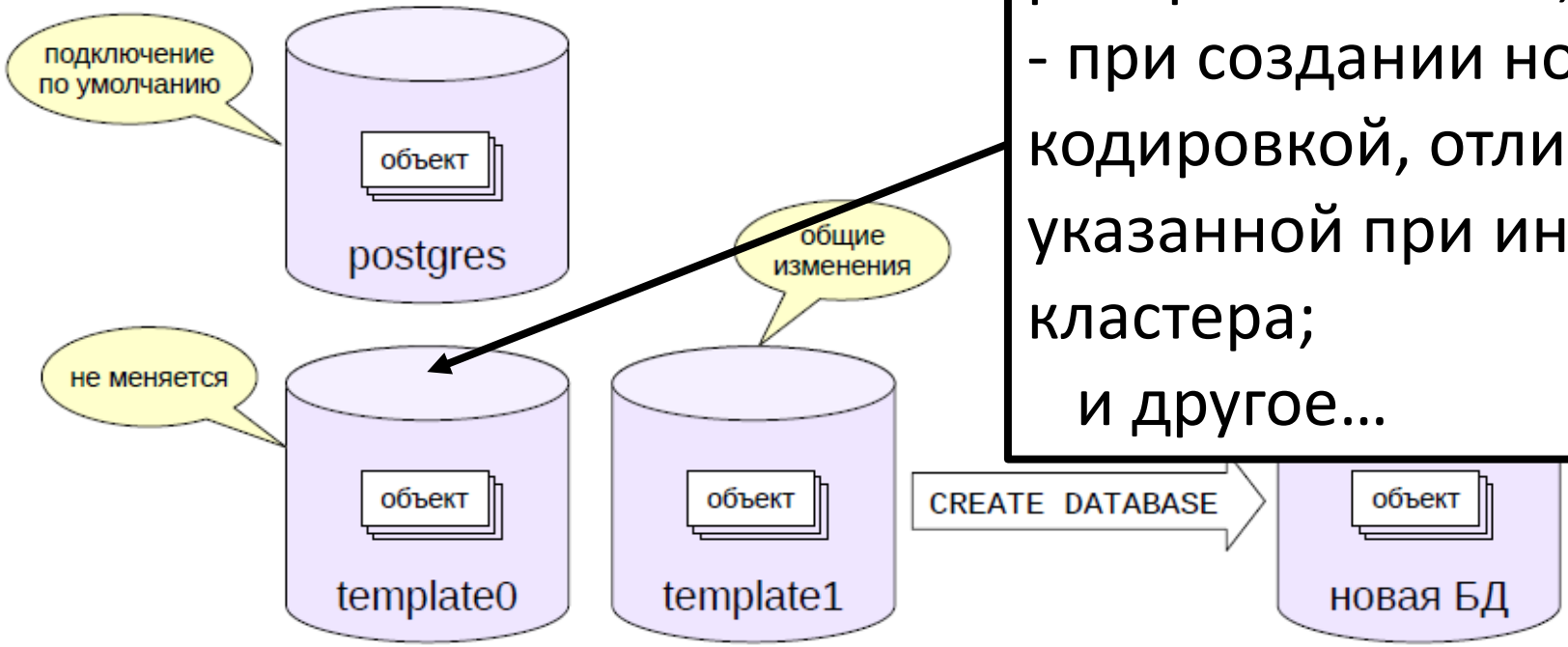


используется по умолчанию для создания новых баз данных.

Кластер баз данных

Инициализация кластера создает три базы данных

Новая база всегда клонируется из су



- для восстановления БД из резервной копии;
- при создании новой БД с кодировкой, отличной от указанной при инициализации кластера;
  и другое…

Базы данных

Список баз данных можно получить в psql такой командой:

```
=> \l
                                                    List of databases
   Name    |  Owner   | Encoding | Locale Provider |   Collate   |    Ctype    | ICU
Locale | ICU Rules |   Access privileges
-----------+----------+----------+-----------------+-------------+-------------+----------
--+-----------+-----------------------
 postgres  | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
  |           |
 student   | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
  |           |
 template0 | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
  |           | =c/postgres          +
           |          |          |                 |             |             |
  |           | postgres=CTc/postgres
 template1 | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
  |           | =c/postgres          +
           |          |          |                 |             |             |
  |           | postgres=CTc/postgres
(4 rows)
```

Когда мы создаем новую базу данных, она (по умолчанию) копируется из шаблона template1.

```
=> CREATE DATABASE data_logical;

CREATE DATABASE

=> \c data_logical

You are now connected to database "data_logical" as user "student".

=> \l

                                                                 List of databases
     Name     |  Owner   | Encoding | Locale Provider |   Collate   |    Ctype    | ICU
Locale | ICU Rules |   Access privileges
--------------+----------+----------+-----------------+-------------+-------------+-------
-----+-----------+-----------------------
 data_logical | student  | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
      |           |
 postgres     | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
      |           |
 student      | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
      |           |
 template0    | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
      |           | =c/postgres          +
              |           |          |                 |             |             |
      |           | postgres=CTc/postgres
 template1    | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
      |           | =c/postgres          +
              |           |          |                 |             |             |
      |           | postgres=CTc/postgres
(5 rows)
```

Схемы

Пространство имен для объектов

- разделение объектов на логические группы
- предотвращение конфликта имен между приложениями

Схема и пользователь — разные сущности

Специальные схемы

- public — по умолчанию в ней создаются все объекты
- pg_catalog — системный каталог
- information_schema — вариант системного каталога
- pg_temp — для временных таблиц
- …

Схемы

Пространство имен для объектов

- разделение объектов на логические группы
- предотвращение конфликта имен между приложениями

Схема и пользователь — разные сущности

Специальные схемы

- public — по умолчанию в ней создаются все объекты
- pg_catalog — системный каталог
- information_schema — вариант системного каталога
- pg_temp — для временных таблиц
- …

Схемы

Пространство имен для объектов

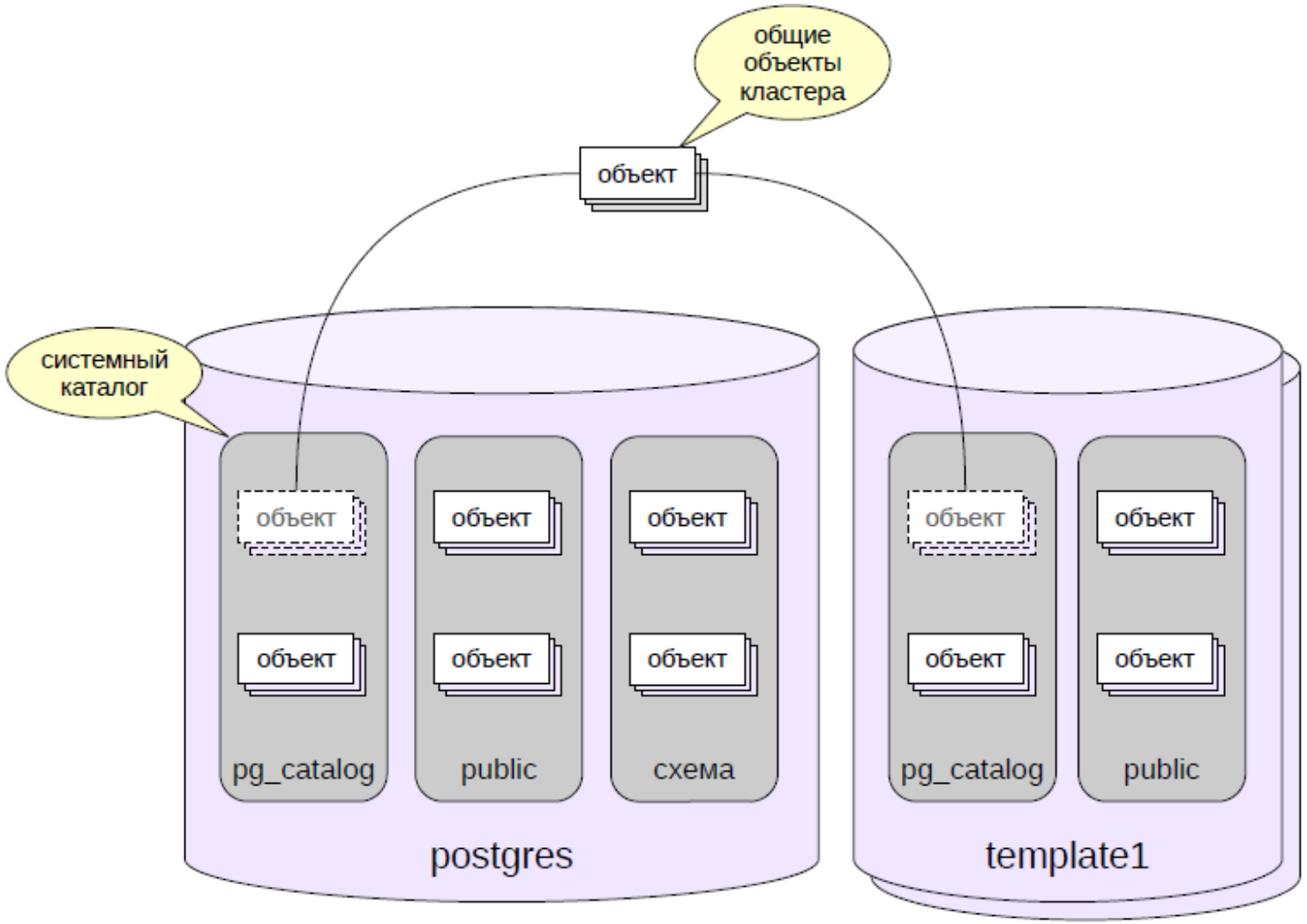
- разделение объектов на логические группы
- предотвращение конфликта имен между приложениями

Схема и пользователь — разные сущности

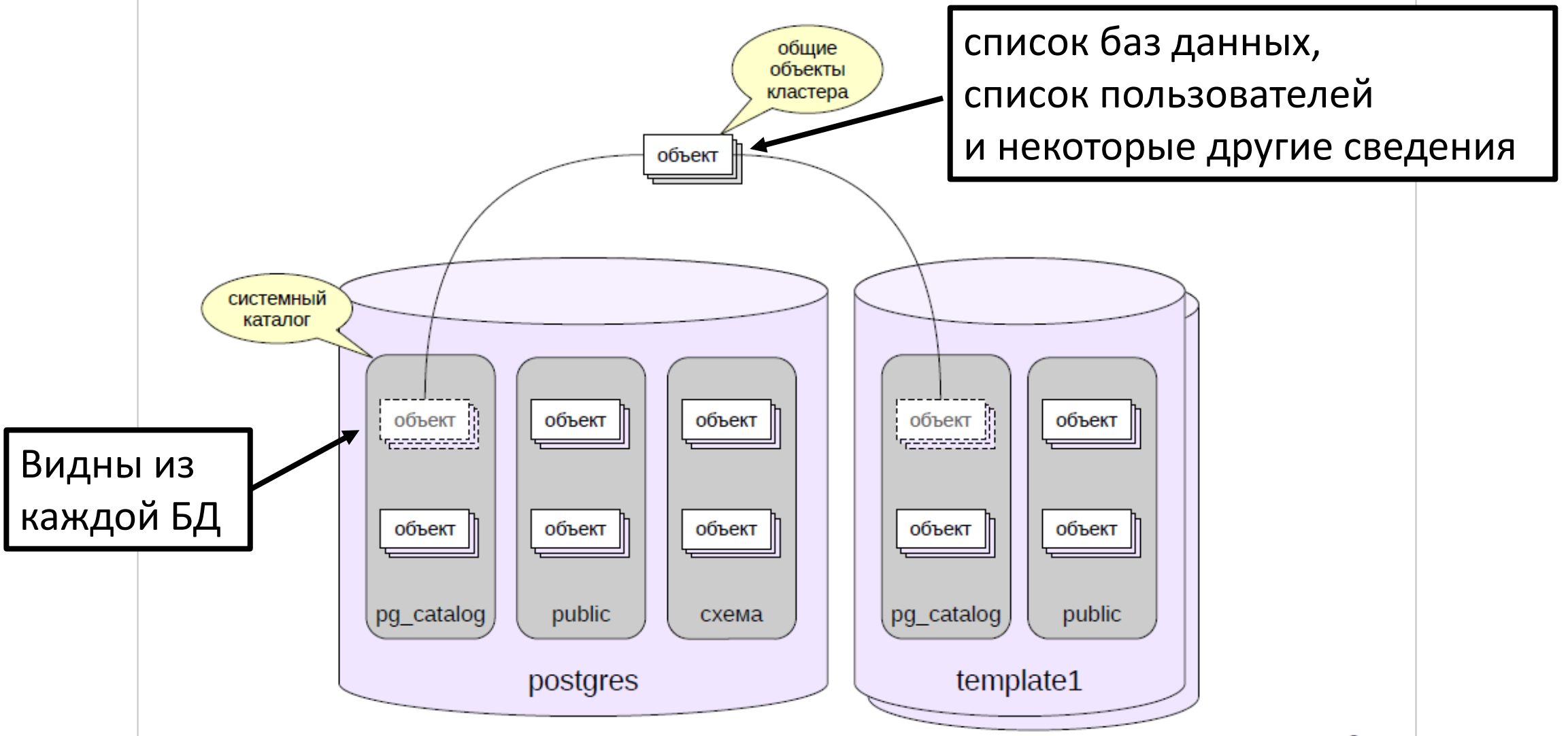
Специальные схемы

- public — по умолчанию в ней создаются все объекты
- pg_catalog — системный каталог
- information_schema — вариант системного каталога
- pg_temp — для временных таблиц
- …

Базы и схемы кластера



Базы и схемы кластера



Схемы

Для вывода списка схем в psql есть специальная команда (\dn = describe namespace):

```
=> \dn
```

```
      List of schemas
  Name  |       Owner
--------+-------------------
 public | pg_database_owner
(1 row)
```

Эта команда не показывает служебные схемы. Чтобы увидеть их, нужно добавить модификатор S (он работает аналогичным образом и для многих других команд):

```
=> \dnS
```

```
            List of schemas
        Name        |       Owner
--------------------+-------------------
 information_schema | postgres
 pg_catalog         | postgres
 pg_toast           | postgres
 public             | pg_database_owner
(4 rows)
```

Еще один полезный модификатор — знак «плюс», который выводит дополнительную информацию:

```
=> \dn+

                                  List of schemas
  Name  |       Owner       |           Access privileges            |      Description
--------+-------------------+----------------------------------------+-------------------
----
 public | pg_database_owner | pg_database_owner=UC/pg_database_owner+| standard public
schema
        |                   | =U/pg_database_owner                   |
(1 row)
```

Создадим новую схему:

```
=> CREATE SCHEMA special;

CREATE SCHEMA

=> \dn

       List of schemas
  Name   |       Owner
---------+-------------------
 public  | pg_database_owner
 special | student
(2 rows)
```

Создадим таблицу:

```
=> CREATE TABLE t(n integer);

CREATE TABLE
```

По умолчанию таблица будет создана в схеме public. Список таблиц в этой схеме можно получить командой \dt с указанием шаблона для имен схем и таблиц:

```
=> \dt public.*

        List of relations
 Schema | Name | Type  |  Owner
--------+------+-------+---------
 public | t    | table | student
(1 row)
```

Таблицу (как и другие объекты) можно перемещать между схемами. Поскольку речь идет о логической организации, перемещение происходит только в системном каталоге; сами данные физически остаются на месте.

```
=> ALTER TABLE t SET SCHEMA special;

ALTER TABLE
```

Что останется в схеме public?

```
=> \dt public.*

Did not find any relation named "public.*".
```

Ничего. А в special?

```
=> \dt special.*

        List of relations
 Schema  | Name | Type  |  Owner
---------+------+-------+---------
 special | t    | table | student
(1 row)
```

Таблица переместилась. Теперь к ней можно обращаться с явным указанием схемы:

```
=> SELECT * FROM special.t;

 n
---
(0 rows)
```

Но если опустить имя схемы, таблица не будет найдена:

```
=> SELECT * FROM t;

ERROR:  relation "t" does not exist
LINE 1: SELECT * FROM t;
                      ^
```

Путь поиска

Определение схемы объекта

- квалифицированное имя (*схема.имя*) явно определяет схему
- имя без квалификатора проверяется в схемах, указанных в пути поиска

Путь поиска

- определяется параметром *search_path*,
  реальное значение — функция current_schemas
- исключаются несуществующие схемы и схемы, к которым нет доступа
- первая явно указанная в пути схема используется для создания объектов
- схемы pg_temp и pg_catalog неявно включаются первыми,
  если не указаны в *search_path*

Путь поиска

Определение схемы объекта

- квалифицированное имя (*схема.имя*) явно определяет схему
- имя без квалификатора проверяется в схемах, указанных в пути поиска

Путь поиска

- определяется параметром *search_path*,
  реальное значение — функция current_schemas
- исключаются несуществующие схемы и схемы, к которым нет доступа
- первая явно указанная в пути схема используется для создания объектов
- схемы pg_temp и pg_catalog неявно включаются первыми,
  если не указаны в *search_path*

включает неявные схемы

Путь поиска

Путь поиска по умолчанию имеет такое значение:

```
=> SHOW search_path;

   search_path
-----------------
 "$user", public
(1 row)
```

Конструкция «$user» обозначает схему с тем же именем, что и имя текущего пользователя (в нашем случае — student). Поскольку такой схемы нет, она игнорируется.

Чтобы не думать над тем, какие схемы есть, каких нет, а какие недоступны, можно воспользоваться функцией:

```
=> SELECT current_schemas(false);

 current_schemas
-----------------
 {public}
(1 row)
```

Передаваемый в функцию логический параметр управляет отображением системных схем, неявно включаемых при поиске. Мы можем увидеть, что кроме исключения несуществующей схемы PostgreSQL неявно включил в начало списка схему системного каталога:

```
=> SELECT current_schemas(true);

   current_schemas
---------------------
 {pg_catalog,public}
(1 row)
```

Установим путь поиска, например, так:

```
=> SET search_path = public, special;

SET
```

Теперь таблица будет найдена.

```
=> SELECT * FROM t;

 n
---
(0 rows)
```

Но параметр можно установить и на уровне отдельной базы данных:

```
=> ALTER DATABASE data_logical SET search_path = public, special;

ALTER DATABASE
```

Теперь он будет устанавливаться для всех новых подключений к БД data_logical. Проверим:

```
=> \c data_logical

You are now connected to database "data_logical" as user "student".

=> SHOW search_path;

   search_path
-----------------
 public, special
(1 row)
```

Но параметр можно установить и на уровне отдельной базы данных:

```
=> ALTER DATABASE data_logical SET search_path = public, special;
ALTER DATABASE
```

Теперь он будет устанавливаться для всех новых подключений к БД data_logical. Проверим:

```
=> \c data_logical

You are now connected to database "data_logical" as user "student".

=> SHOW search_path;

   search_path
-----------------
 public, special
(1 row)
```

Системный каталог

Описание всех объектов кластера

- набор таблиц в каждой базе данных (схема pg_catalog) и несколько глобальных объектов кластера
- набор представлений для удобства

Доступ

- запросы SQL, специальные команды psql

Правила организации

- названия таблиц начинаются с pg_
- имена столбцов содержат трехбуквенный префикс
- в качестве ключа используется столбец oid типа oid
- названия объектов хранятся в нижнем регистре

Системный каталог

Описание всех объектов кластера

- набор таблиц в каждой базе данных (схема pg_
  и несколько глобальных объектов кластера
- набор представлений для удобства

Метаинформация о разных объектах (таблицах, индексах, функциях и т.д.)

Доступ

- запросы SQL, специальные команды psql

Правила организации

- названия таблиц начинаются с pg_
- имена столбцов содержат трехбуквенный префикс
- в качестве ключа используется столбец oid типа oid
- названия объектов хранятся в нижнем регистре

Системный каталог

Для того, чтобы вывести информацию о любых объектах, psql (как и другие интерактивные пользовательские средства) обращается к таблицам системного каталога.

Например, команда \l для получения списка баз данных кластера, обращается к таблице:

```
=> SELECT datname FROM pg_database;

   datname
--------------
 postgres
 student
 template1
 template0
 data_logical
(5 rows)
```

Мы всегда можем посмотреть, какие запросы выполняет команда:

```
=> \set ECHO_HIDDEN on

=> \l

******** QUERY **********
SELECT
  d.datname as "Name",
  pg_catalog.pg_get_userbyid(d.datdba) as "Owner",
  pg_catalog.pg_encoding_to_char(d.encoding) as "Encoding",
  CASE d.datlocprovider WHEN 'c' THEN 'libc' WHEN 'i' THEN 'icu' END AS "Locale Provider",
  d.datcollate as "Collate",
  d.datctype as "Ctype",
  d.daticulocale as "ICU Locale",
  d.daticurules as "ICU Rules",
  pg_catalog.array_to_string(d.datacl, E'\n') AS "Access privileges"
FROM pg_catalog.pg_database d
ORDER BY 1;
**************************

                                                    List of databases
     Name     |  Owner   | Encoding | Locale Provider |   Collate   |    Ctype    | ICU
Locale | ICU Rules |   Access privileges
--------------+----------+----------+-----------------+-------------+-------------+-------
-----+-----------+-----------------------
 data_logical | student  | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
     |           |
 postgres     | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
     |           |
 student      | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
     |           |
 template0    | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
     |           | =c/postgres          +
              |          |          |                 |             |             |
     |           | postgres=CTc/postgres
 template1    | postgres | UTF8     | libc            | en_US.UTF-8 | en_US.UTF-8 |
     |           | =c/postgres          +
              |          |          |                 |             |             |
     |           | postgres=CTc/postgres
(5 rows)
```

Таким образом можно исследовать системный каталог.

Отключим вывод команд.

```
=> \set ECHO_HIDDEN off
```

Список схем находится в таблице:

```
=> SELECT nspname FROM pg_namespace;


      nspname
--------------------
 pg_toast
 pg_catalog
 public
 information_schema
 special
(5 rows)
```

А на такие объекты, как таблицы и индексы, можно посмотреть так:

```
=> SELECT relname, relkind, relnamespace FROM pg_class WHERE relname = 't';

 relname | relkind | relnamespace
---------+---------+--------------
 t       | r       |        16391
(1 row)
```

Все столбцы здесь начинаются на rel (relation, отношение).

- relkind — тип объекта (r — таблица, i — индекс и т. п.);
- relnamespace — схема.

А на такие объекты, как таблицы и индексы, можно посмотреть так:

```
=> SELECT relname, relkind, relnamespace FROM pg_class WHERE relname = 't';

 relname | relkind | relnamespace
---------+---------+--------------
 t       | r       |        16391
(1 row)
```

Все столбцы здесь начинаются на rel (relation, отношение).

- relkind — тип объекта (r — таблица, i — индекс и т. п.);
- relnamespace — схема.

Поле relnamespace имеет тип oid; вот соответствующая строка таблицы pg_namespace:

```
=> SELECT oid, nspname FROM pg_namespace WHERE oid = 16391;

  oid  | nspname
-------+---------
 16391 | special
(1 row)
```

А на такие объекты, как таблицы и индексы, можно посмотреть так:

```
=> SELECT relname, relkind, relnamespace FROM pg_class WHERE relname = 't';

 relname | relkind | relnamespace
---------+---------+--------------
 t       | r       |        16391
(1 row)
```

Все столбцы здесь начинаются на rel (relation, отношение).

- relkind — тип объекта (r — таблица, i — индекс и т. п.);
- relnamespace — схема.

Поле relnamespace имеет тип oid; вот соответствующая строка таблицы pg_namespace:

```
=> SELECT oid, nspname FROM pg_namespace WHERE oid = 16391;

  oid  | nspname
-------+---------
 16391 | special
(1 row)
```

Для удобства преобразования между текстовым представлением и oid можно воспользоваться приведением к специальному типу-псевдониму regnamespace:

```
=> SELECT relname, relkind, relnamespace::regnamespace::text
FROM pg_class WHERE relname = 't';

 relname | relkind | relnamespace
---------+---------+--------------
 t       | r       | special
(1 row)
```

А вот как можно получить список объектов в схеме, например, pg_catalog:

```
=> SELECT relname, relkind FROM pg_class
WHERE relnamespace = 'pg_catalog'::regnamespace LIMIT 5;

            relname              | relkind
---------------------------------+---------
 pg_statistic                    | r
 pg_type                         | r
 pg_foreign_table                | r
 pg_proc_oid_index               | i
 pg_proc_proname_args_nsp_index | i
(5 rows)
```

Аналогичные reg-типы определены и для некоторых других таблиц системного каталога. Они позволяют упростить запросы и обойтись без явного соединения таблиц.

Удаление объектов

Можно ли удалить схему special?

```
=> DROP SCHEMA special;
```

```
ERROR:  cannot drop schema special because other objects depend on it
DETAIL:  table t depends on schema special
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
```

Схему нельзя удалить, если в ней находятся какие-либо объекты. Сначала надо удалить или перенести их.

Удаление объектов

Можно ли удалить схему special?

```
=> DROP SCHEMA special;

ERROR:  cannot drop schema special because other objects depend on it
DETAIL:  table t depends on schema special
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
```

Схему нельзя удалить, если в ней находятся какие-либо объекты. Сначала надо удалить или перенести их.

Но можно удалить схему сразу вместе со всеми ее объектами:

```
=> DROP SCHEMA special CASCADE;

  NOTICE:  drop cascades to table t
  DROP SCHEMA
```

А что с удалением базы данных целиком? Во-первых, нельзя удалить базу, к которой вы подключены в данный момент, поэтому отключимся от нее.

```
=> \conninfo

You are connected to database "data_logical" as user "student" via socket in
"/var/run/postgresql" at port "5432".

=> \c postgres

You are now connected to database "postgres" as user "student".
```

А что с удалением базы данных целиком? Во-первых, нельзя удалить базу, к которой вы подключены в данный момент, поэтому отключимся от нее.

```
=> \conninfo

You are connected to database "data_logical" as user "student" via socket in
"/var/run/postgresql" at port "5432".

=> \c postgres

You are now connected to database "postgres" as user "student".
```

Во-вторых, базу данных также нельзя удалить, если к ней есть активные подключения. Создадим такое подключение в отдельном сеансе и попробуем удалить ее:

```
=> \c data_logical

You are now connected to database "data_logical" as user "student".

=> DROP DATABASE data_logical;

ERROR:  database "data_logical" is being accessed by other users
DETAIL:  There is 1 other session using the database.
```

Получили ошибку. Однако можно вызвать команду удаления с параметром FORCE, тогда она будет пытаться принудительно завершить все подключения к БД, а затем удалит ее:

```
=> DROP DATABASE data_logical WITH (FORCE);

DROP DATABASE
```

Итоги

Логически

- кластер содержит базы данных,
- базы данных — схемы,
- схемы — конкретные объекты (таблицы, индексы и т. п.)

Базы данных создаются клонированием существующих

Схема объекта определяется по пути поиска

Полное описание содержимого кластера баз данных хранится в системном каталоге

Организация данных
Физическая структура



Темы

Табличные пространства и каталоги

Файлы и страницы данных

Слои: данные, карты видимости и свободного пространства

Технология TOAST

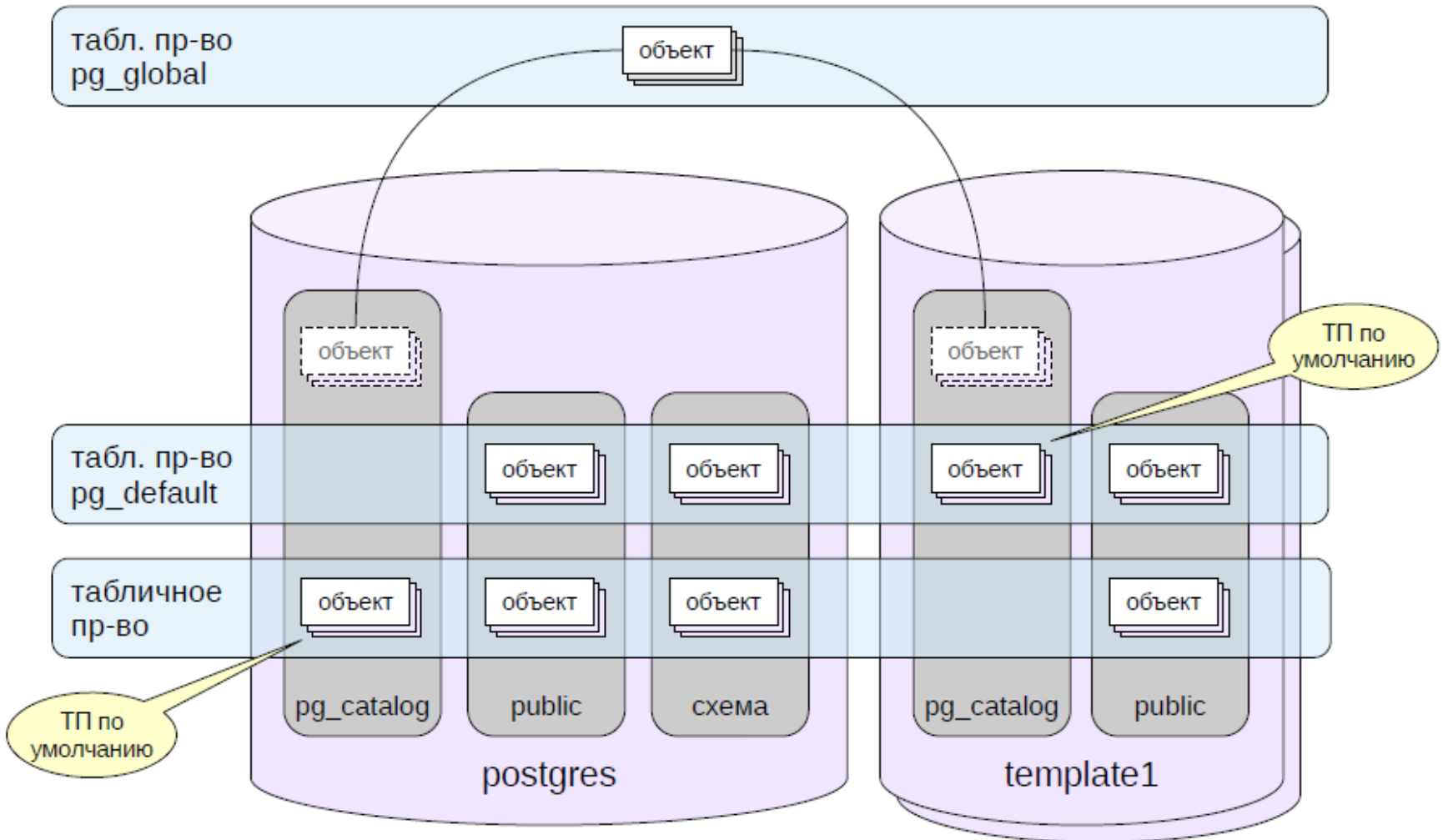
Табличные пространства

физическое хранение данных –
расположение данных в файловой системе

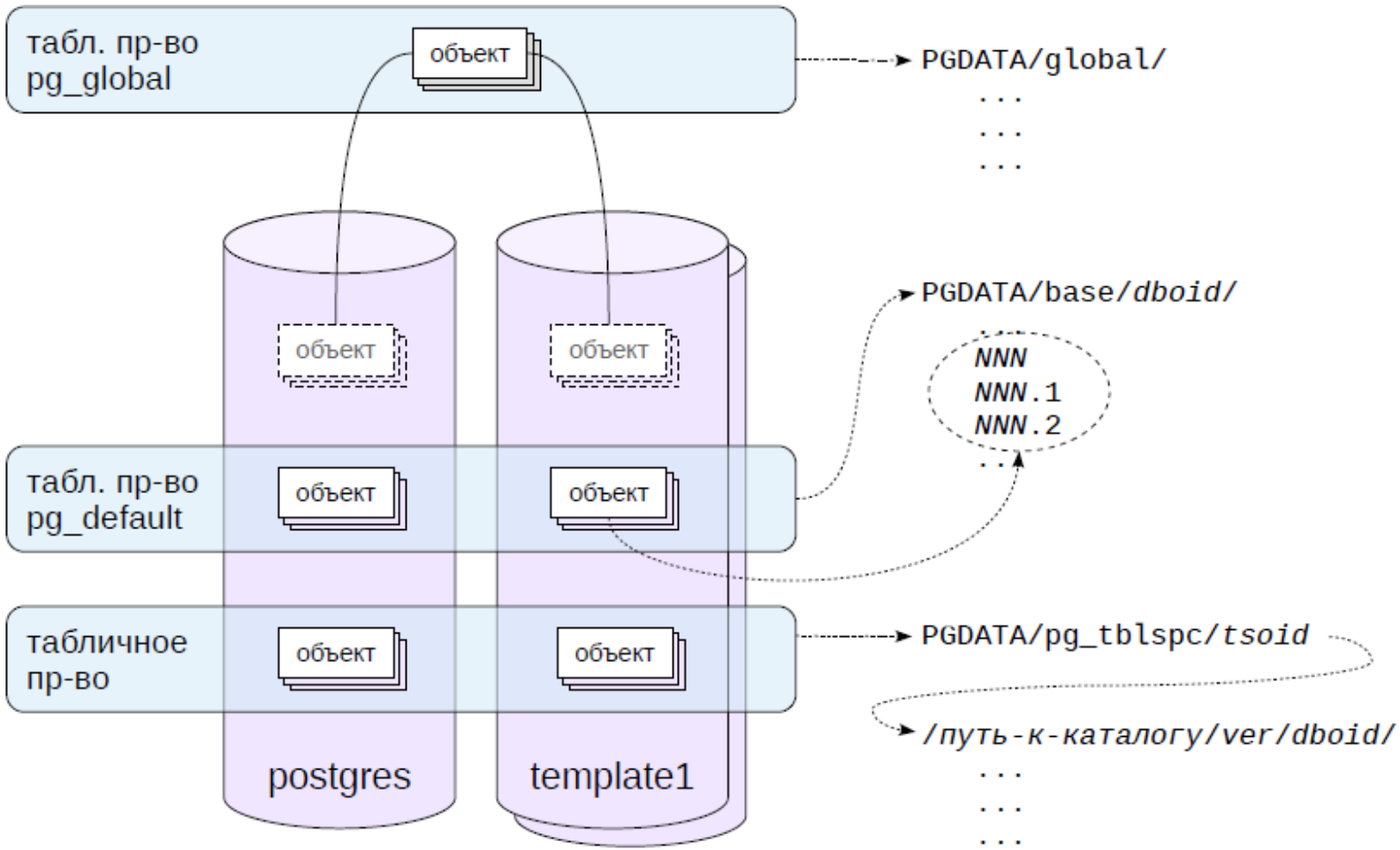
- Например, можно создать одно ТП на медленных дисках для архивных данных,

- а другое – на быстрых дисках для данных, с которыми идет активная работа

Табличные пространства

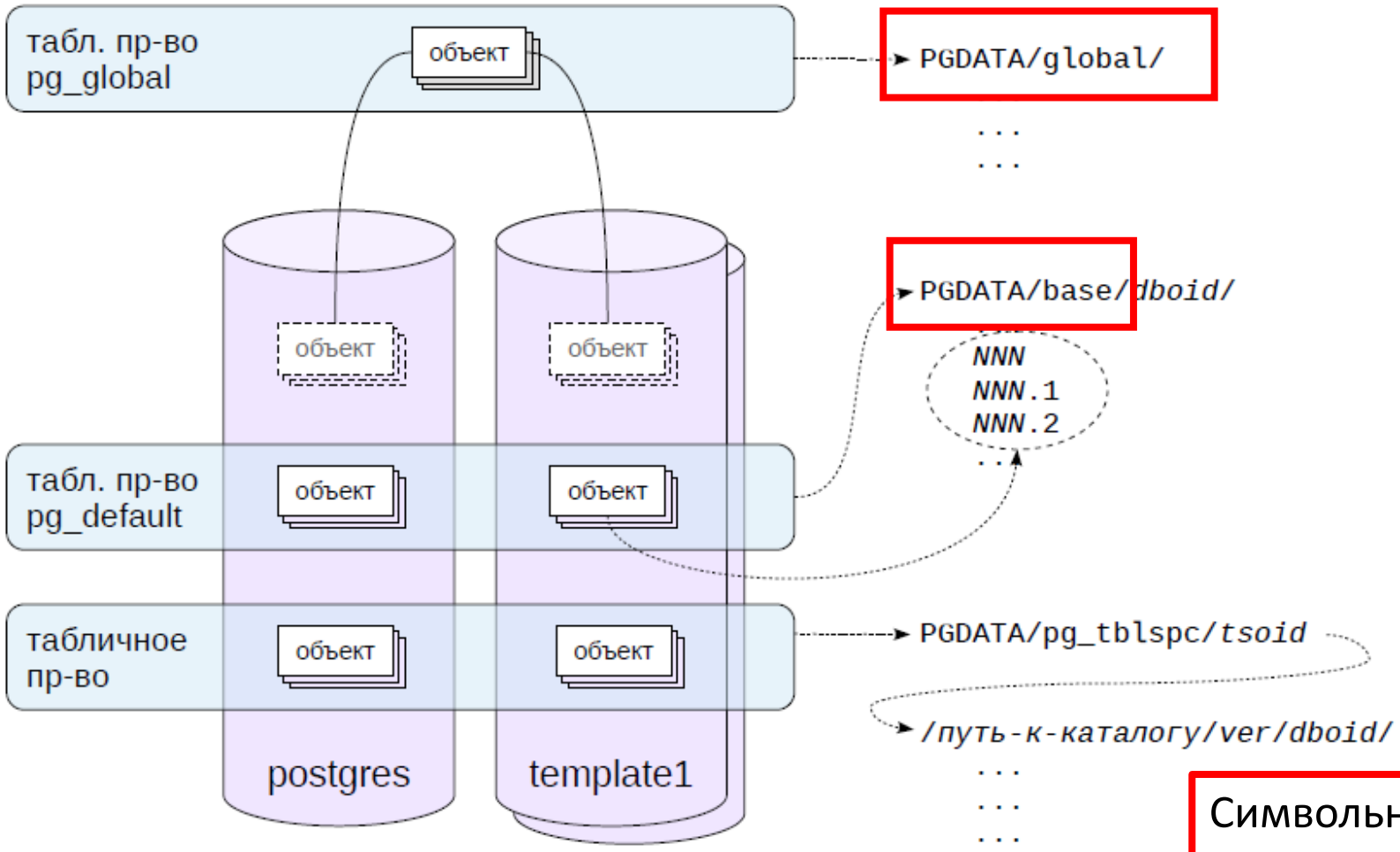


Каталоги


табл. пр-во
pg_global
объект
PGDATA/global/
. . .
. . .
. . .
PGDATA/base/dboid/
. . .
NNN
NNN.1
NNN.2
. . .
табл. пр-во
pg_default
объект
объект
объект
объект
табличное
пр-во
объект
объект
PGDATA/pg_tblspc/tsoid
/путь-к-каталогу/ver/dboid/
. . .
. . .
. . .
postgres
template1

Каталоги

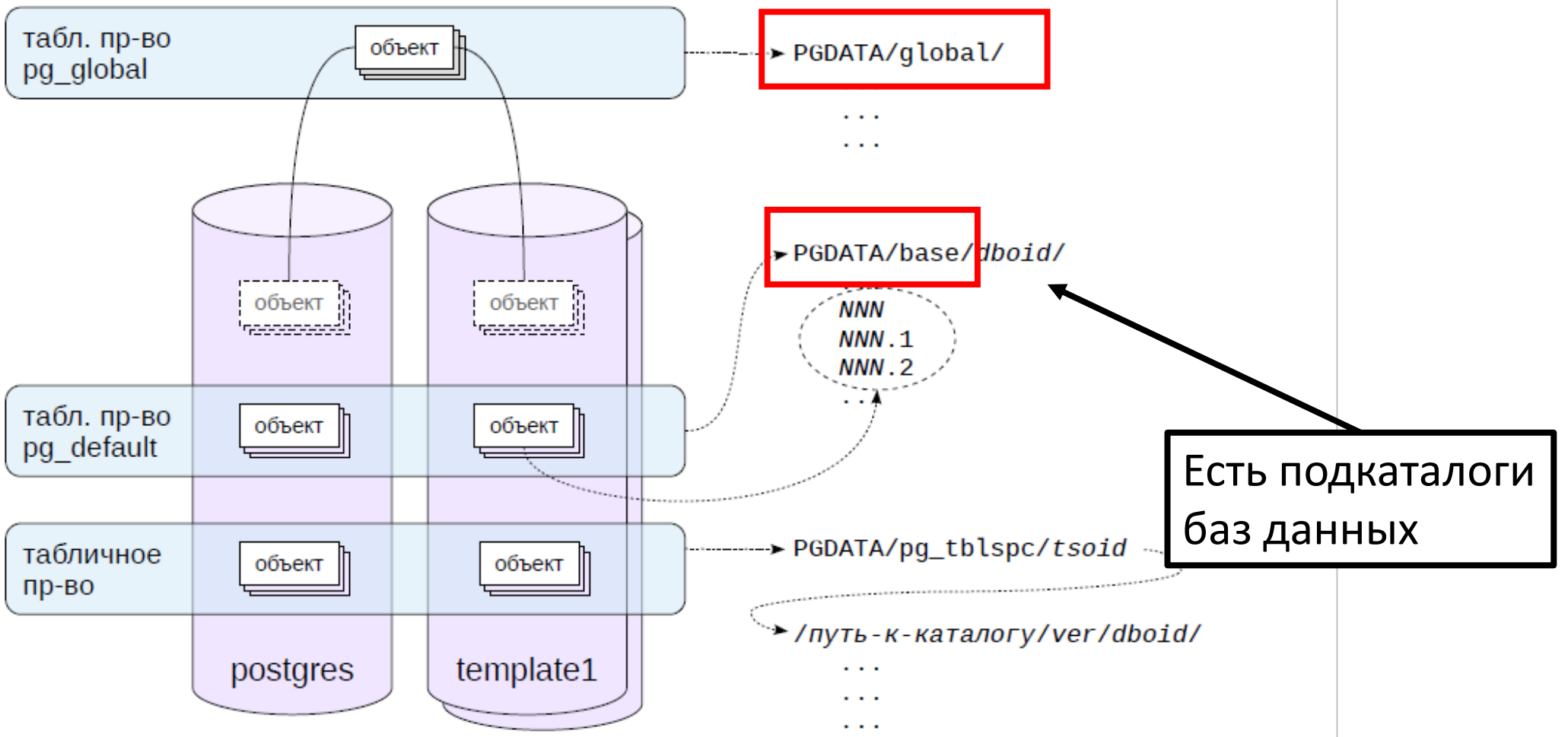
табличное пространство — это указание на каталог, в котором располагаются данные



Символьная ссылка в каталоге PGDATA/pg_tblspc/.

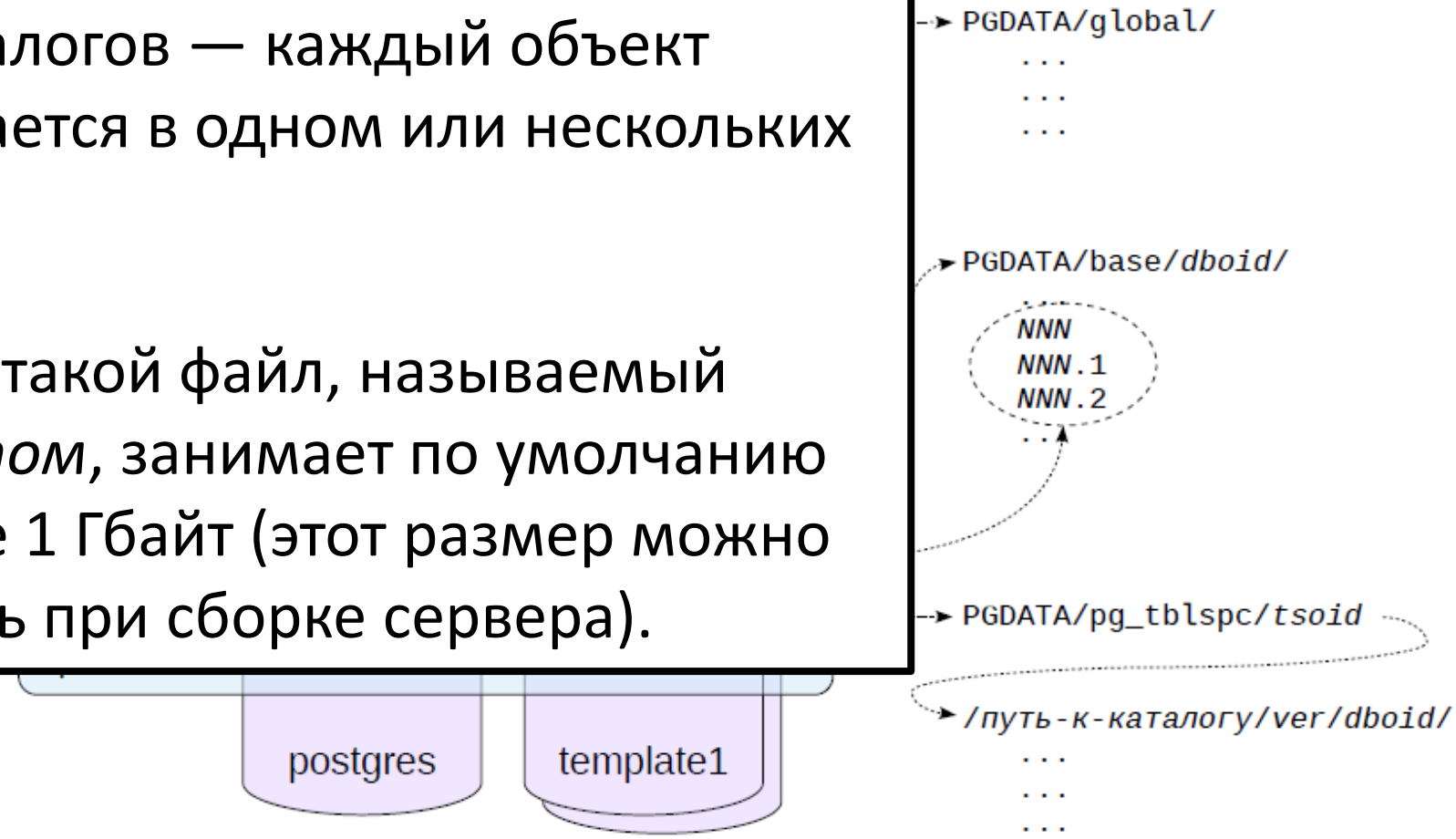
Каталоги

табличное пространство — это указание на каталог, в котором располагаются данные



Каталоги

- Объекты хранятся в файлах внутри этих каталогов — каждый объект размещается в одном или нескольких файлах.
- Каждый такой файл, называемый *сегментом*, занимает по умолчанию не более 1 Гбайт (этот размер можно изменить при сборке сервера).



Использование табличных пространств

Изначально в кластере присутствуют два табличных пространства. Информация о них содержится в системном каталоге:

```
=> SELECT spcname FROM pg_tablespace;

  spcname
------------
 pg_default
 pg_global
(2 rows)
```

Конечно, это одна из глобальных для всего кластера таблиц.

Аналогичная команда psql:

```
=> \db

       List of tablespaces
    Name    |  Owner   | Location
------------+----------+----------
 pg_default | postgres |
 pg_global  | postgres |
(2 rows)
```

Для нового табличного пространства нужен пустой каталог, владельцем которого является пользователь ОС, запускающий сервер СУБД:

```
=> \! sudo mkdir /var/lib/postgresql/ts_dir
```

Сменим владельца каталога:

```
=> \! sudo chown postgres /var/lib/postgresql/ts_dir
```

Теперь можем выполнить команду создания табличного пространства:

```
=> CREATE TABLESPACE ts LOCATION '/var/lib/postgresql/ts_dir';

CREATE TABLESPACE

=> \db

                List of tablespaces
    Name    |  Owner   |          Location
------------+----------+----------------------------
 pg_default | postgres |
 pg_global  | postgres |
 ts         | student  | /var/lib/postgresql/ts_dir
(3 rows)
```

Обязательно указывается путь (каталог)

При создании базы данных можно указать табличное пространство по умолчанию:

```
=> CREATE DATABASE data_physical TABLESPACE ts;

CREATE DATABASE

=> \c data_physical

You are now connected to database "data_physical" as user "student".
```

Это означает, что все объекты базы по умолчанию будут создаваться в этом табличном пространстве.

```
=> CREATE TABLE t(id integer PRIMARY KEY, s text);

CREATE TABLE

=> INSERT INTO t(id, s)
    SELECT id, id::text FROM generate_series(1,100_000) id;

INSERT 0 100000
```

Слои и файлы для объекта

Основной

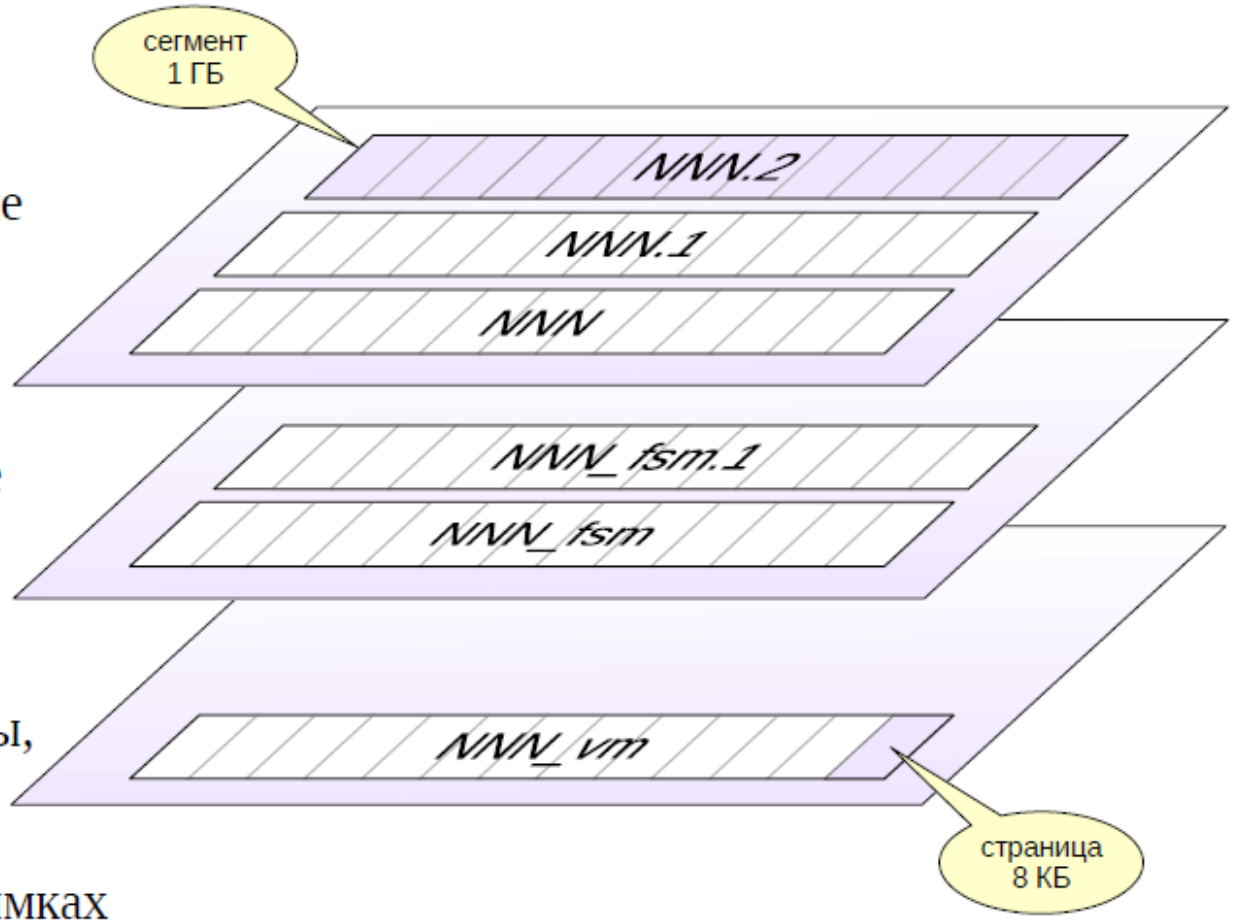
собственно данные

Карта свободного пространства

в страницах после очистки

Карта видимости

отмечает страницы, на которых все версии строк видны во всех снимках



Слои и файлы (forks)

Основной

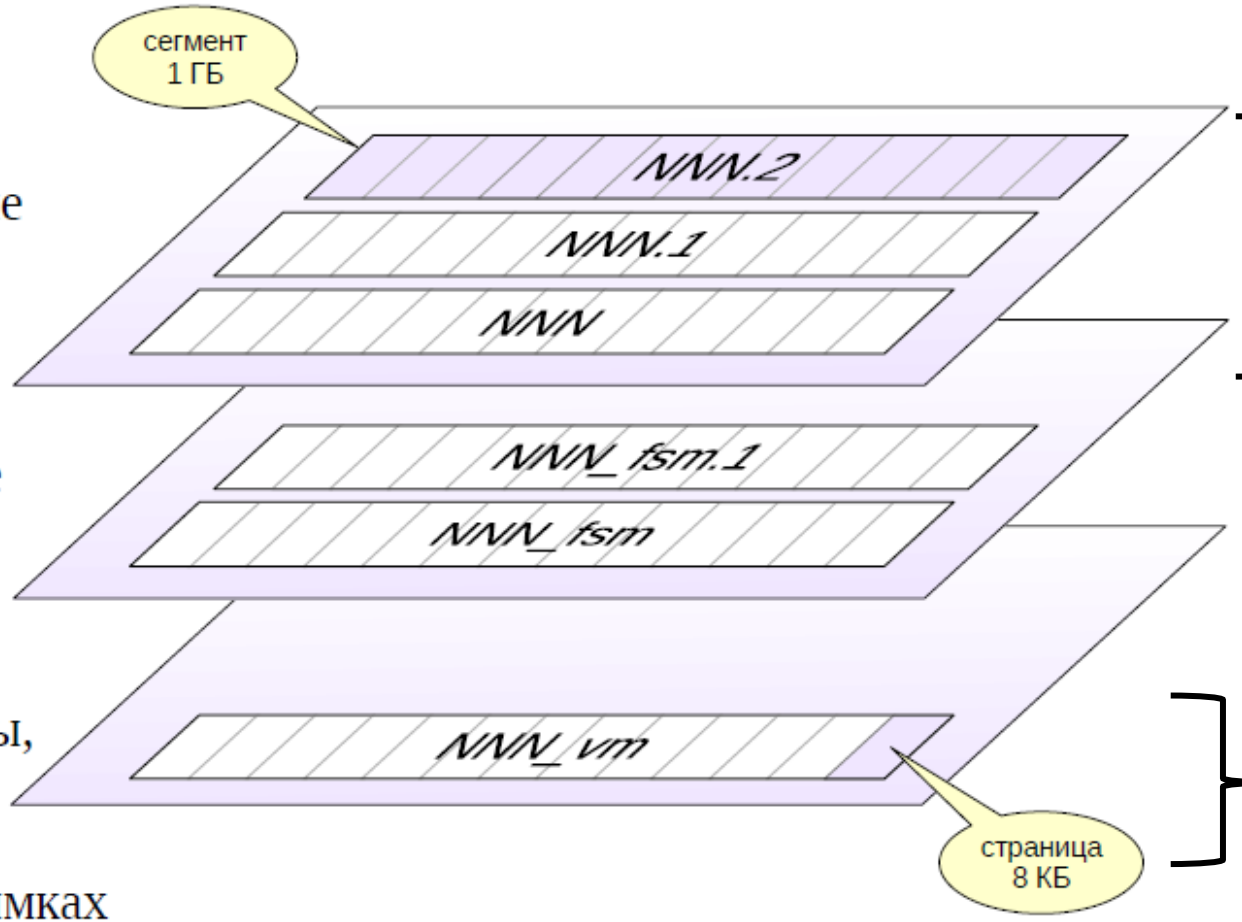
собственно данные

Карта свободного пространства

в страницах после очистки

Карта видимости

отмечает страницы, на которых все версии строк видны во всех снимках



Слой – набор сегментов

Сегмент – один или несколько файлов, логически разбит на страницы

Слои и файлы

Основной

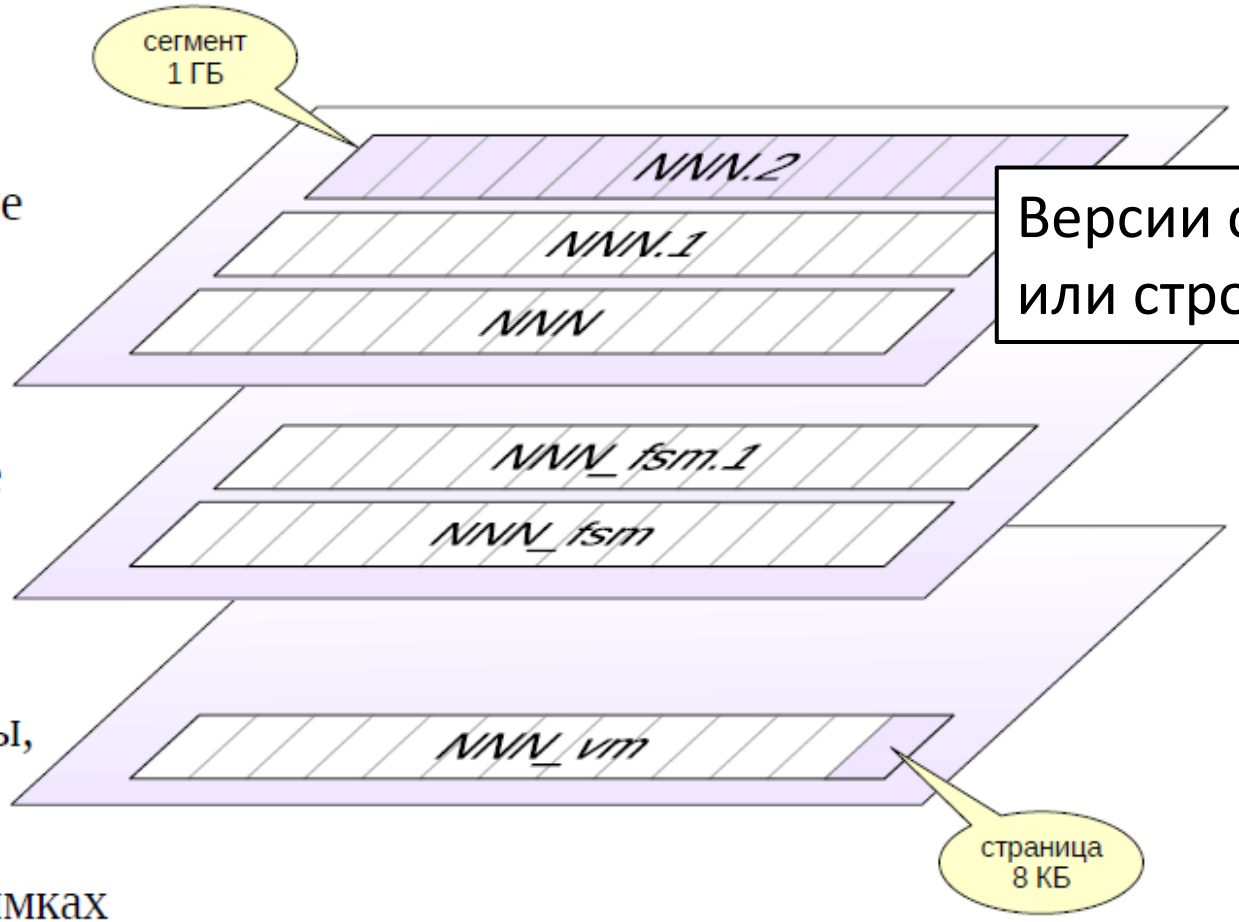
собственно данные

Карта свободного пространства

в страницах после очистки

Карта видимости

отмечает страницы, на которых все версии строк видны во всех снимках



Версии строк таблиц или строки индексов

Слои и файлы

Основной

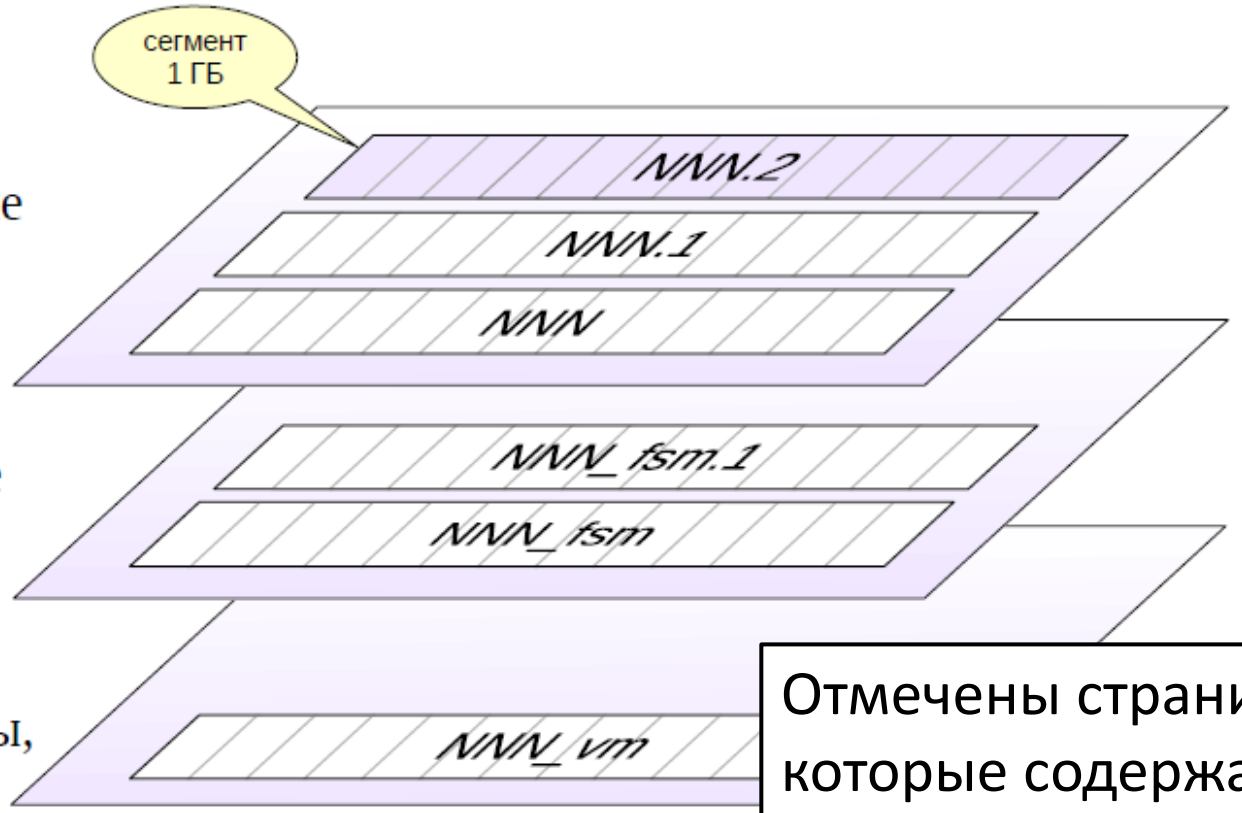
собственно данные

Карта свободного пространства

в страницах после очистки

Карта видимости

отмечает страницы, на которых все версии строк видны во всех снимках



vm (visibility map)

Отмечены страницы, которые содержат только актуальные версии строк, причем эти версии видны во всех снимках данных

Слои и файлы

fsm (free space map)

Основной

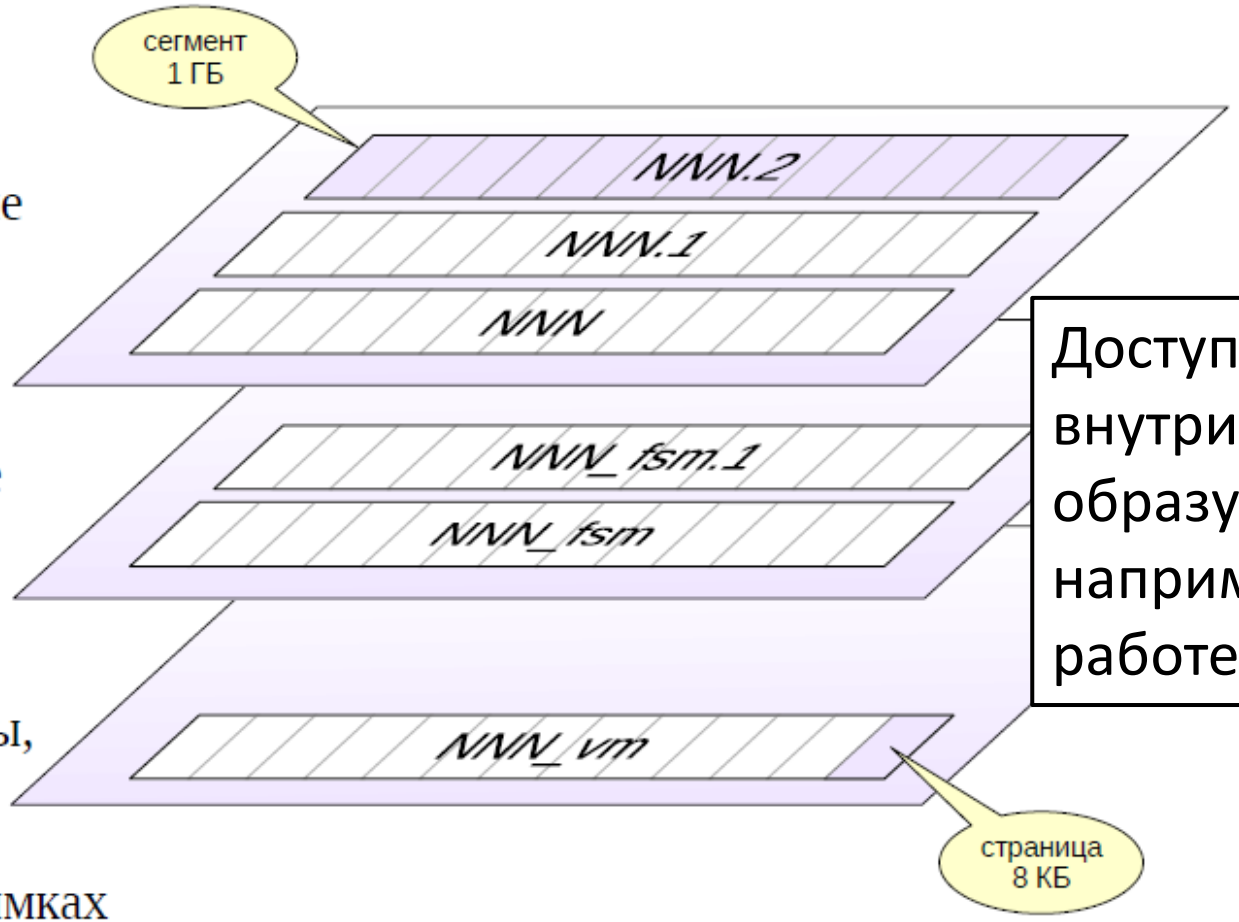
собственно данные

Карта свободного пространства

в страницах после очистки

Карта видимости

отмечает страницы, на которых все версии строк видны во всех снимках



Доступное место внутри страниц, образующееся, например, при работе очистки

Слои и файлы

fsm (free space map)

Основной

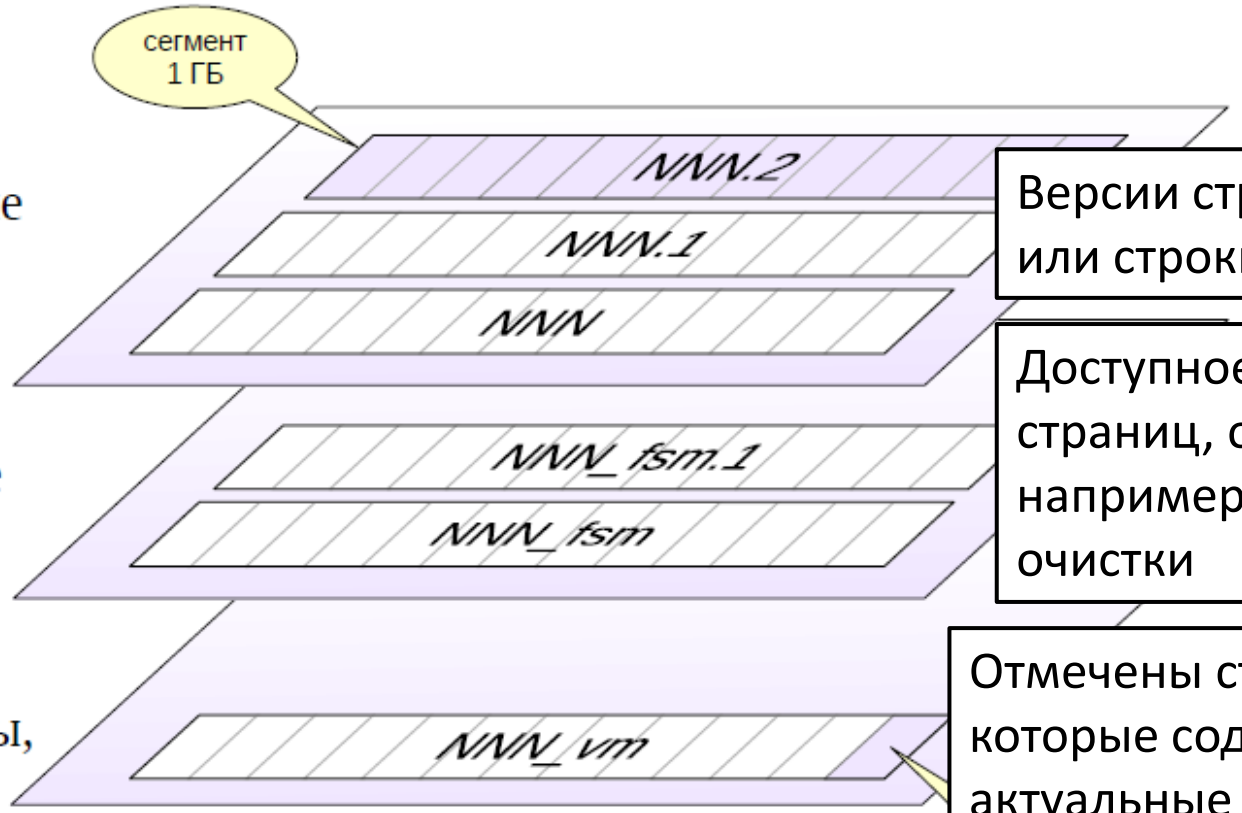
собственно данные

Карта свободного пространства

в страницах после очистки

Карта видимости

отмечает страницы, на которых все версии строк видны во всех снимках



Версии строк таблиц или строки индексов

Доступное место внутри страниц, образующееся, например, при работе очистки

Отмечены страницы, которые содержат только актуальные версии строк, причем эти версии видны во всех снимках данных

Слои и файлы

Очистка обеспечит нам создание всех слоев таблицы:

```
=> VACUUM t;

VACUUM
```

Узнать расположение файлов, из которых состоит объект, можно так:

```
=> SELECT pg_relation_filepath('t');

            pg_relation_filepath
-----------------------------------------------
 pg_tblspc/16390/PG_16_202307071/16391/16392
(1 row)
```

Посмотрим на сами файлы (имя и размер в байтах):

```
student$ sudo bash -c 'cd /var/lib/postgresql/16/main/pg_tblspc/16390/PG_16_202307071/16391; ls -l
16392*'

-rw------- 1 postgres postgres 4423680 июн 24 04:31 16392
-rw------- 1 postgres postgres   24576 июн 24 04:31 16392_fsm
-rw------- 1 postgres postgres    8192 июн 24 04:31 16392_vm
```

Видно, что они относятся к трем слоям: основному, fsm и vm.

Объекты можно перемещать между табличными пространствами, но (в отличие от схем) это приводит к физическому перемещению данных:

```
=> ALTER TABLE t SET TABLESPACE pg_default;

ALTER TABLE

=> SELECT pg_relation_filepath('t');

 pg_relation_filepath
----------------------
 base/16391/16399
(1 row)
```

Размер объектов

Узнать размер, занимаемый базой данных и объектами в ней, можно с помощью ряда функций.

```
=> SELECT pg_database_size('data_physical');

 pg_database_size
------------------
         14488035
(1 row)
```

Для упрощения восприятия можно вывести число в отформатированном виде:

```
=> SELECT pg_size_pretty(pg_database_size('data_physical'));

 pg_size_pretty
----------------
 14 MB
(1 row)
```

Полный размер таблицы (вместе со всеми индексами):

```
=> SELECT pg_size_pretty(pg_total_relation_size('t'));

 pg_size_pretty
----------------
 6568 kB
(1 row)
```

А также отдельно размер таблицы...

```
=> SELECT pg_size_pretty(pg_table_size('t'));
```

```
 pg_size_pretty
----------------
 4360 kB
(1 row)
```

...и индексов:

```
=> SELECT pg_size_pretty(pg_indexes_size('t'));
```

```
 pg_size_pretty
----------------
 2208 kB
(1 row)
```

При желании можно узнать и размер отдельных слоев таблицы, например:

```
=> SELECT pg_size_pretty(pg_relation_size('t','main'));

 pg_size_pretty
----------------
 4320 kB
(1 row)
```

Объем, который занимает на диске табличное пространство, показывает другая функция:

```
=> SELECT pg_size_pretty(pg_tablespace_size('ts'));

 pg_size_pretty
----------------
 9804 kB
(1 row)
```

TOAST

The Oversized Attributes Storage Technique

Версия строки должна помещаться на одну страницу

- можно сжать часть атрибутов,
- или вынести в отдельную TOAST-таблицу,
- или сжать и вынести одновременно

TOAST-таблица

- схема pg_toast
- поддержана собственным индексом
- «длинные» атрибуты разделены на части размером меньше страницы
- читается только при обращении к «длинному» атрибуту
- собственная версионность
- работает прозрачно для приложения

TOAST

Добавим в таблицу очень длинную строку:

```
=> INSERT INTO t(id, s)
SELECT 0, string_agg(id::text,'.') FROM generate_series(1,5000) AS id;

INSERT 0 1
```

Изменится ли размер таблицы?

```
=> SELECT pg_size_pretty(pg_table_size('t'));

 pg_size_pretty
----------------
 4416 kB
(1 row)
```

Да. А размер основного слоя, в котором хранятся данные?

```
=> SELECT pg_size_pretty(pg_relation_size('t','main'));

 pg_size_pretty
----------------
 4320 kB
(1 row)
```

Нет.

Поскольку версия строки не помещается в одну страницу, значение атрибута s будет разрезано на части и помещено в отдельную toast-таблицу. Ее можно отыскать в системном каталоге (мы используем тип regclass, чтобы преобразовать oid в имя отношения):

```
=> SELECT oid, reltoastrelid::regclass::text FROM pg_class WHERE relname='t';

  oid  |      reltoastrelid
-------+-------------------------
 16392 | pg_toast.pg_toast_16392
(1 row)
```

Символьная строка хранится по частям, из которых PostgreSQL при необходимости склеивает полное значение:

```
=> SELECT chunk_id, chunk_seq, left(chunk_data::text,45) AS chuck_data
FROM pg_toast.pg_toast_16392;

 chunk_id | chunk_seq |                  chuck_data
----------+-----------+----------------------------------------------
    16402 |         0 | \x545d000000312e322e332e342e00352e362e372e382
    16402 |         1 | \x392e353161ff31002e3531322e353133002e3531342
    16402 |         2 | \xe215e216e217e218abe219e11a30e11b30e11c30e11
    16402 |         3 | \x11f4aa3611f43611f43611f43611f4aa3611f43611f
    16402 |         4 | \xf43211f4325511f43211f43211f43211f4325511f43
    16402 |         5 | \xf43811f43811f43811f4385511f43811f43811f4381
    16402 |         6 | \x11f4aa3411f43411f43411f43411f4aa3411f43411f
    16402 |         7 | \x0132370132aa370132370132370132370132aa38013
    16402 |         8 | \xf43611f43611f43611f4365511f43611f43611f4361
(9 rows)
```

Итоги

Физически

- данные распределены по табличным пространствам (каталогам)
- объект представлен несколькими слоями
- каждый слой состоит из одного или нескольких файлов-сегментов

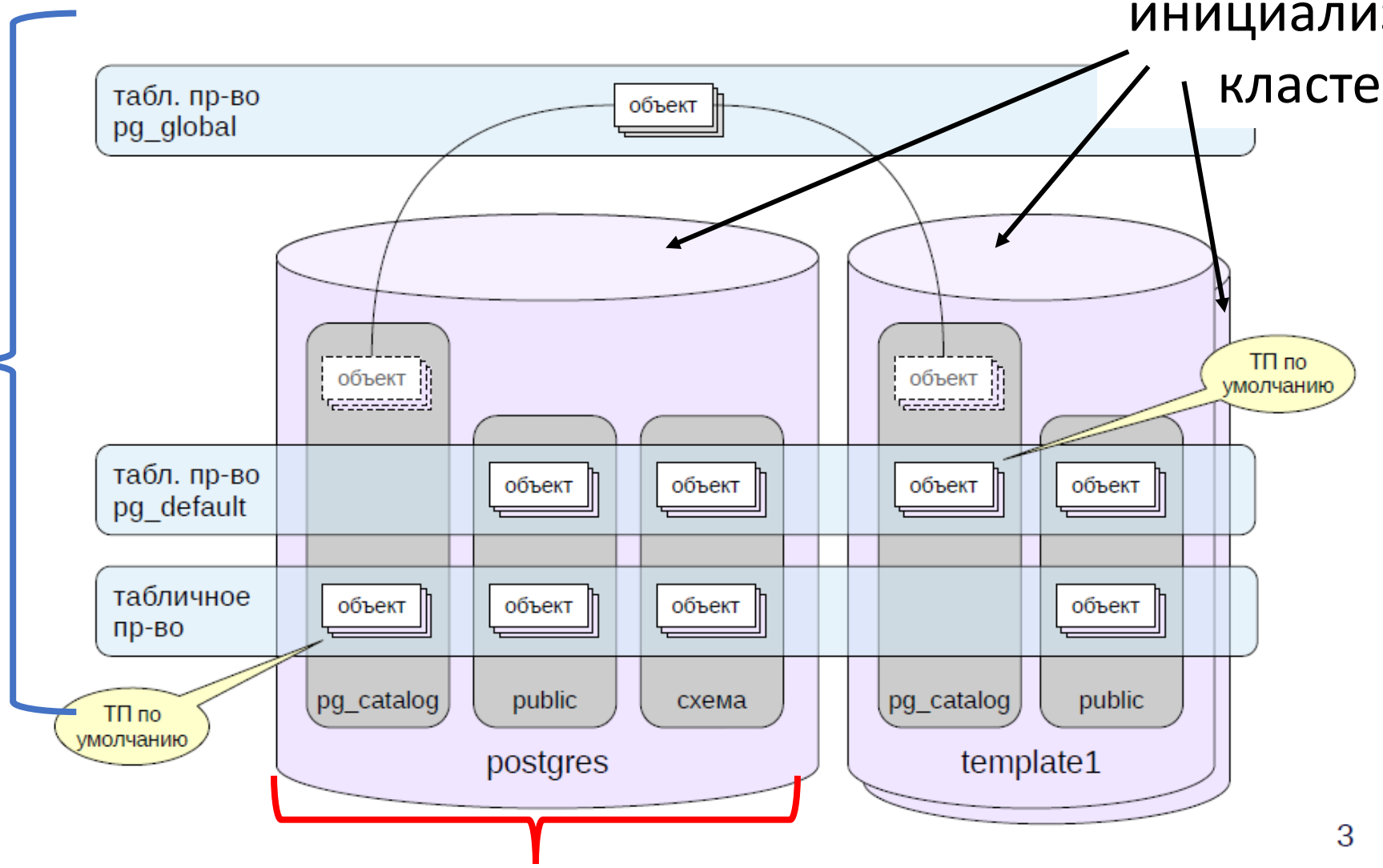
Табличными пространствами управляет администратор

Слои, файлы, TOAST — внутренняя кухня PostgreSQL

Табличные пространства

Три БД при инициализации кластера

ФИЗИЧЕСКАЯ ОРГАНИЗАЦИЯ ДАННЫХ - табличные пространства



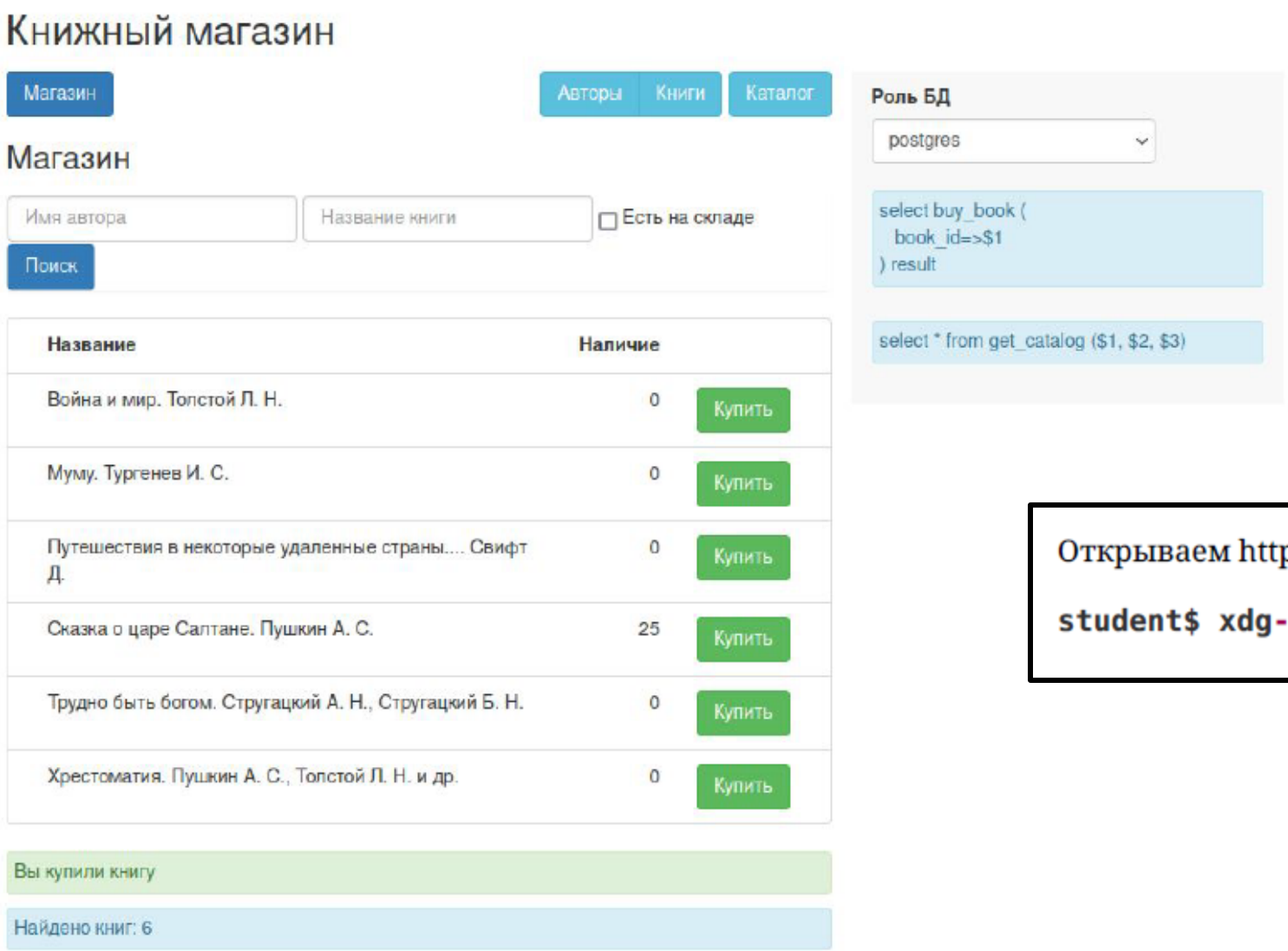
ЛОГИЧЕСКАЯ ОРГАНИЗАЦИЯ ДАННЫХ - схемы

Учебное приложение

Книжный магазин



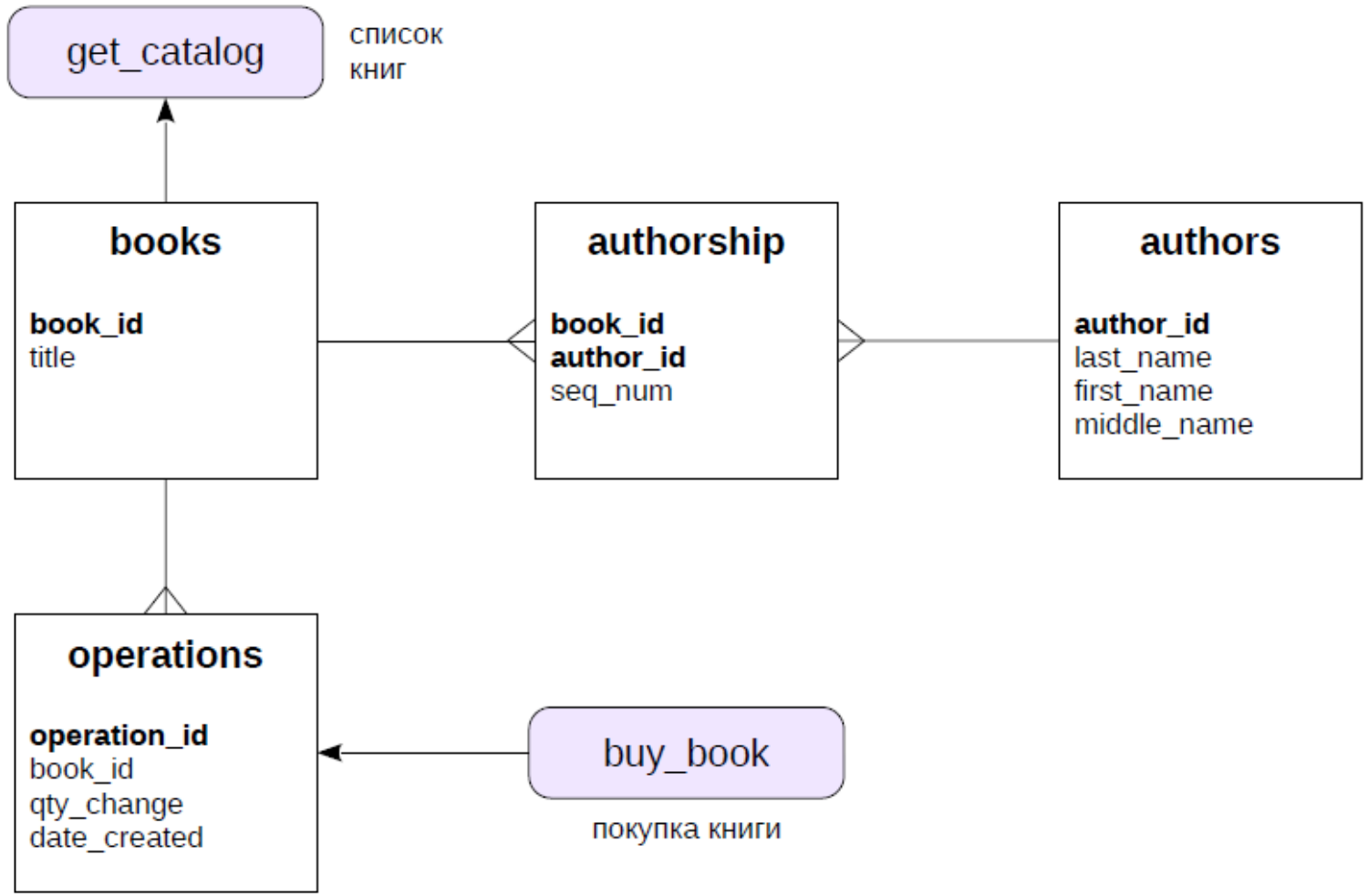
Приложение



Открываем http://localhost...

```
student$ xdg-open http://localhost
```

Интерфейс покупателя



Интерфейс сотрудника

